



AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI Conquer 2026 – Topic Team

Week 02 Progress Report: Baseline Replication and Mathematical Parameter Formulation of Vision MLPs (ResMLP & RepMLP) on CIFAR-100

Debug Team

Nguyễn Thị Diễm My Lê Tiến Anh Nguyễn Mạnh Tân

Hoàng Nguyễn Nhật Minh Lê Minh Hoàng

Abstract

This report presents the Week 02 progress of **Debug Team** in the **AI VIET NAM (AIO2026) Topic Team** program. Following our project plan from Week 01, our main goal this week was to replicate and benchmark two baseline Vision MLP models: **ResMLP** (*Touvron et al.*, 2021) and **RepMLP** (*Ding et al.*, 2022).

To make the comparison fair, we matched both models to around $\approx 2.16\text{M}$ – 2.19M parameters and trained them from scratch on **CIFAR-100** for 100 epochs using the same setup (AdamW, cosine learning rate schedule, RandAugment, Mixup, CutMix, and FP16 mixed precision). We also calculate the exact parameter formulas for both models, explaining the difference between ResMLP’s single-scale columnar structure (2, 191, 204 parameters) and RepMLP’s multi-stage structure (2, 155, 684 parameters).

Our results show that RepMLP learns faster in early epochs and reaches higher accuracy (71.46% vs. 62.40%), mainly because its convolutional branches help capture local image patterns. These results give us a solid baseline to test our proposed re-parameterized token mixer in Week 03.

Table of Contents

1	Overview and Week 02 Objectives	3
1.1	Context and project goals	3
1.2	Week 02 main objectives	3
1.3	Scope of this report	3
2	Experimental Setup & Mathematical Parameter Formulation	3
2.1	Unified experimental pipeline	3
2.2	Mathematical derivation of learnable parameters in ResMLP	4
2.3	Mathematical derivation of learnable parameters in RepMLP	6
2.4	Controlled evaluation and fair comparison	8
3	Replication Results & Training Dynamics	8
3.1	Quantitative benchmark results	8
3.2	Convergence dynamics across 100 epochs	8
3.3	Computational efficiency and training time	10
4	Architectural Comparison	11
4.1	Isotropic columnar vs. hierarchical multi-stage architectures	11
4.2	How spatial tokens interact	11
4.3	Linear additivity and re-parameterization	12
5	Technical Challenges, Week 03 Roadmap & TA Consultation	12
5.1	Technical challenges resolved in Week 02	12
5.2	Week 03 roadmap	13
5.3	Questions for Teaching Assistant (TA) consultation	13
6	References	15

1. Overview and Week 02 Objectives

1.1. Context and project goals

In the **AI VIET NAM (AIO2026) Topic Team** program, Weeks 02 and 03 focus on running experiments and gathering results. Our project aims to improve data efficiency in Vision MLPs using structural re-parameterization.

Before testing any new architectures, we first need to implement and test the baseline models properly under the exact same conditions. Therefore, our focus for **Week 02** is simply **replicating baselines and standardizing experiments**.

1.2. Week 02 main objectives

To build a solid starting point for our project, our team focused on three key goals in Week 02:

1. **Replicating baseline architectures:** Implement two representative Vision MLP models from scratch:
 - **ResMLP** [1]: A simple columnar architecture where token mixing uses a single linear layer without non-linear activations.
 - **RepMLP** [2]: A multi-stage architecture that adds parallel depthwise conv layers during training to inject local image details.
2. **Standardizing parameter budgets:** Keep both models at approximately $\approx 2.16\text{M} - 2.19\text{M}$ parameters so that the comparison on **CIFAR-100** is fair. We also write down the exact closed-form formulas for their parameter counts.
3. **Setting up a shared training pipeline:** Train both models for 100 epochs from scratch with identical hyperparameters, data augmentations (RandAugment, Mixup, CutMix), and FP16 mixed precision, recording their loss and accuracy curves.

1.3. Scope of this report

Although we have started coding other experimental variants in our repository (such as local stems, pyramid ResMLP, and our proposed Rep-ResMixer), we deliberately keep them out of this Week 02 report.

Our team wants to first confirm the baseline results and ensure all training logs are solid before evaluating new modifications in Week 03.

2. Experimental Setup & Mathematical Parameter Formulation

2.1. Unified experimental pipeline

To make sure our comparison is completely fair, both models are trained with the exact same training pipeline implemented in `src/engine/trainer.py`. We set a fixed random seed of 42

across Python, NumPy, and PyTorch backends for full reproducibility.

- **Dataset and split:** We use CIFAR-100, which contains 60,000 color images of size 32×32 across 100 classes (500 training images per class). This data-limited setting is well suited for testing data efficiency, because vision models without spatial inductive bias often suffer from data hunger when trained from scratch. We split the 50,000 images into 45,000 images for training and 5,000 images for validation, reserving the remaining 10,000 images for final testing.
- **Data augmentation:** Pure MLPs have high representational capacity but lack built-in spatial invariance, making them prone to memorizing training samples. We use RandAugment ($N = 2, M = 9$), Random Erasing ($p = 0.1$), and Color Jitter (0.1) to introduce geometric and color variations. Furthermore, we apply Mixup ($\alpha = 0.8$) and CutMix ($\alpha = 1.0$) to blend pairs of images and labels, which encourages the network to learn smoother decision boundaries and prevents overfitting to specific pixel patterns.
- **Optimizer and schedule:** We train both models for 100 epochs using AdamW with an initial learning rate of 10^{-3} , weight decay of 0.05, and momentum parameters $(\beta_1, \beta_2) = (0.9, 0.999)$. AdamW decouples weight decay from gradient updates, providing stable regularization across linear layers and normalization scales. The learning rate decays smoothly down to 10^{-6} using a Cosine Annealing schedule, allowing broad exploration of the loss surface early on and fine convergence into stable minima near the end.
- **Loss function and precision:** We use cross-entropy loss with label smoothing ($\epsilon = 0.1$) to soften one-hot targets, keeping the network from becoming overconfident on fine-grained class boundaries. Gradient clipping at maximum norm 1.0 prevents sudden gradient spikes. We also enable PyTorch Automatic Mixed Precision (AMP FP16), which cuts memory usage in half and accelerates training on GPU Tensor Cores.

2.2. Mathematical derivation of learnable parameters in ResMLP

ResMLP [1] uses an isotropic columnar structure where the token count N and channel dimension D stay constant across all L layers. For an input image $\mathbf{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$ with $C_{\text{in}} = 3$ and $H = W = 32$, the image is divided into non-overlapping patches of size $P \times P$ ($P = 4$):

$$N = \left(\frac{H}{P}\right) * \left(\frac{W}{P}\right) = \left(\frac{32}{4}\right)^2 = 64 \quad \text{tokens.} \quad (1)$$

Our configuration uses embedding dimension $D = 256$, depth $L = 8$, FFN expansion factor $E = 2$, and $K = 100$ classes. We calculate the exact parameter count for each component below:

2.2.1. Patch projection stem

The patch embedding uses a 2D convolution with kernel size $P = 4$ and stride $P = 4$, mapping from $C_{\text{in}} = 3$ to $D = 256$ channels:

$$P_{\text{stem}} = \underbrace{(C_{\text{in}} * P^2) * D}_{\text{Conv weights}} + \underbrace{D}_{\text{Bias}} = (3 * 16) * 256 + 256 = 12,288 + 256 = \mathbf{12,544}. \quad (2)$$

2.2.2. ResMLP block (repeated $L = 8$ times)

Each ResMLP block contains two residual sublayers:

- (a) **Token-mixing sublayer:** Tokens communicate along the sequence dimension N , shared across all channels D :

- Two Affine layers (scale $\alpha \in \mathbb{R}^D$ and bias $\beta \in \mathbb{R}^D$): $2 * 2D = 4D = 4 * 256 = 1,024$.
- Spatial Linear layer $\text{Linear}(N, N)$ with weight matrix $\mathbf{W}_{\text{token}} \in \mathbb{R}^{N \times N}$ and bias $\mathbf{b}_{\text{token}} \in \mathbb{R}^N$:

$$N^2 + N = 64^2 + 64 = 4,096 + 64 = 4,160. \quad (3)$$

Total parameters for the token-mixing sublayer per block:

$$P_{\text{token}} = 4D + N^2 + N = 1,024 + 4,160 = \mathbf{5,184}. \quad (4)$$

- (b) **Channel-mixing sublayer (FFN):** Projects channel dimensions from D to $E * D$ and back:

- Two Affine layers: $2 * 2D = 4D = 1,024$.
- Linear expansion $\text{Linear}(D, E * D)$: $D * (E * D) + E * D = 2 * 256^2 + 2 * 256 = 131,072 + 512 = 131,584$.
- GELU activation: 0 parameters.
- Linear projection $\text{Linear}(E * D, D)$: $(E * D) * D + D = 2 * 256^2 + 256 = 131,072 + 256 = 131,328$.

Total parameters for the channel-mixing sublayer per block:

$$P_{\text{channel}} = 4D + 2 * E * D^2 + (E + 1) * D = 1,024 + 262,144 + 768 = \mathbf{263,936}. \quad (5)$$

Adding both sublayers gives the total parameters per ResMLP block:

$$P_{\text{block}} = P_{\text{token}} + P_{\text{channel}} = 5,184 + 263,936 = \mathbf{269,120}. \quad (6)$$

For the entire backbone of $L = 8$ blocks:

$$P_{\text{backbone}} = L * P_{\text{block}} = 8 * 269,120 = \mathbf{2,152,960}. \quad (7)$$

2.2.3. Classification head

Global average pooling over the N tokens uses 0 parameters. The classification head is a single layer $\text{Linear}(D, K)$ mapping to $K = 100$ classes:

$$P_{\text{head}} = D * K + K = 256 * 100 + 100 = 25,600 + 100 = \mathbf{25,700}. \quad (8)$$

Submodule / layer	Component details	Parameters	Percentage (%)
Patch embedding	Conv2d(3, 256, 4, stride = 4) with bias	12,544	0.57%
ResMLP blocks ($L = 8$)			
– Token-mixing sublayers	$8 * [4D + N^2 + N] = 8 * 5,184$	41,472	1.89%
– Channel-mixing sublayers (FFN)	$8 * [4D + 2ED^2 + (E + 1)D] = 8 * 263,936$	2,111,488	96.36%
Classification head	Linear(256, 100) with bias	25,700	1.17%
Total parameters		2,191,204	100.00%

Table 1: Layer-by-layer parameter breakdown of ResMLP ($D = 256, L = 8, E = 2$).

2.2.4. Total parameters in ResMLP

Combining all parts, the total parameter count of ResMLP is given by:

$$P_{\text{ResMLP}} = (C_{\text{in}} * P^2 + 1) * D + L * [2 * E * D^2 + (E + 9) * D + N^2 + N] + (K * D + K) \quad (9)$$

Plugging in our values ($C_{\text{in}} = 3, P = 4, D = 256, N = 64, E = 2, L = 8, K = 100$):

$$P_{\text{ResMLP}} = 12,544 + 2,152,960 + 25,700 = \mathbf{2,191,204} \quad (\approx \mathbf{2.19M}). \quad (10)$$

2.3. Mathematical derivation of learnable parameters in RepMLP

Unlike ResMLP’s single-resolution design, RepMLP [2] uses a multi-stage architecture with local spatial partitions and re-parameterization. For an input image of size 32×32 , the model processes features through a stem followed by three progressive stages:

- **Channel dimensions:** $\mathbf{C} = [C_0, C_1, C_2] = [64, 128, 256]$.
- **Stage depths:** $\mathbf{B} = [B_0, B_1, B_2] = [2, 4, 2]$ blocks.
- **Sharesets:** $\mathbf{S} = [S_0, S_1, S_2] = [1, 4, 16]$.
- **Feature map resolution:** Stage 0 (16×16), Stage 1 (8×8), Stage 2 (4×4).

During training, each RepMLP block includes:

- (a) **Global perceptron (GP):** Squeeze-and-excitation channel attention.
- (b) **Partition-based FC3:** Fully connected layer operating on local $h \times w$ partitions with S sharesets.

- (c) **Auxiliary conv branches:** Parallel 1×1 and 3×3 depthwise convolutions with Batch-Norm to add local image features during training.
- (d) **Feed-forward network (FFN):** Two 1×1 convolutions with expansion factor 4 and GELU activation.

The parameter count during training is calculated as follows:

2.3.1. Patch embedding stem

The stem uses a Conv2d(3, 64, kernel = 2, stride = 2) without bias, followed by BatchNorm2d(64):

$$P_{\text{stem}} = \underbrace{(3 * 2 * 2) * 64}_{\text{Conv weights}} + \underbrace{2 * 64}_{\text{BN}} = 768 + 128 = \mathbf{896}. \quad (11)$$

2.3.2. RepMLP stages and transition layers

Each stage $i \in \{0, 1, 2\}$ contains B_i RepMLP units, followed by a stride-2 transition convolution (for stages $0 \rightarrow 1$ and $1 \rightarrow 2$):

- **Stage 0** ($C_0 = 64, B_0 = 2, S_0 = 1$):
 - Two RepMLP units: 202,688 parameters.
 - Transition layer ($64 \rightarrow 128$, kernel 2, stride 2 + BN): $(128 * 64 * 4) + (2 * 128) = 32,768 + 256 = \mathbf{33,024}$.
 - Stage 0 total: $202,688 + 33,024 = \mathbf{235,712}$.
- **Stage 1** ($C_1 = 128, B_1 = 4, S_1 = 4$):
 - Four RepMLP units: 630,656 parameters.
 - Transition layer ($128 \rightarrow 256$, kernel 2, stride 2 + BN): $(256 * 128 * 4) + (2 * 256) = 131,072 + 512 = \mathbf{131,584}$.
 - Stage 1 total: $630,656 + 131,584 = \mathbf{762,240}$.
- **Stage 2** ($C_2 = 256, B_2 = 2, S_2 = 16$):
 - Two RepMLP units: $\mathbf{1,130,624}$ parameters (final stage, no transition needed).

2.3.3. Classification head

The head uses a final BatchNorm2d(256) followed by global average pooling and a linear layer Linear(256, 100):

$$P_{\text{head}} = \underbrace{2 * 256}_{\text{BN}} + \underbrace{(256 * 100 + 100)}_{\text{Linear}} = 512 + 25,700 = \mathbf{26,212}. \quad (12)$$

2.3.4. Total parameters in RepMLP

Adding up the stem, all three stages, transitions, and the classification head:

$$P_{\text{RepMLP}} = 896 + 235,712 + 762,240 + 1,130,624 + 26,212 = \mathbf{2,155,684} \quad (\approx \mathbf{2.16M}). \quad (13)$$

Submodule / stage	Component details	Parameters	Percentage (%)
Patch embedding stem	Conv2d(3, 64, 2, stride = 2) + BN(64)	896	0.04%
Stage 0 ($16 \times 16, C = 64$)	2 RepMLP units (202, 688) + Transition $64 \rightarrow 128$ (33, 024)	235,712	10.93%
Stage 1 ($8 \times 8, C = 128$)	4 RepMLP units (630, 656) + Transition $128 \rightarrow 256$ (131, 584)	762,240	35.36%
Stage 2 ($4 \times 4, C = 256$)	2 RepMLP units (final stage, no transition)	1,130,624	52.45%
Classification head	BatchNorm2d(256) + Linear(256, 100) with bias	26,212	1.22%
Total parameters		2,155,684	100.00%

Table 2: Layer-by-layer parameter breakdown of RepMLP ($C = [64, 128, 256]$, $B = [2, 4, 2]$).

2.4. Controlled evaluation and fair comparison

As shown in the formulas above, ResMLP (2, 191, 204 parameters) and RepMLP (2, 155, 684 parameters) are matched closely in size (only 1.62% difference). Because both models use virtually the same parameter budget ($\approx 2.16\text{M} - 2.19\text{M}$), training length (100 epochs), and augmentations, any performance difference comes directly from their architectures.

3. Replication Results & Training Dynamics

3.1. Quantitative benchmark results

Both baseline models were trained from scratch on CIFAR-100 for 100 epochs following the setup in Section 2. We recorded training loss, training accuracy, validation loss, Top-1 accuracy, and Top-5 accuracy for every epoch into repository logs (`runs/res_mlp.log`, `runs/resmlp.csv`, and `runs/rep_mlp.log`).

Table 3 summarizes both training and validation metrics from the 100-epoch benchmark.

Model	Architecture	Params	Final Train Acc	Epoch 20 Val Acc	Best Val Acc@1	Best Val Acc@5	Train time
ResMLP	Isotropic Columnar	2,191,204	44.72%	45.52%	62.40%	87.52%	54.65 min
RepMLP	Hierarchical Multi-stage	2,155,684	63.43%	55.92%	71.46%	90.96%	107.13 min

Table 3: Benchmark comparison between ResMLP and RepMLP baselines on CIFAR-100 (100 epochs, seed 42).

3.2. Convergence dynamics across 100 epochs

Looking at both training and validation trajectories across the 100 epochs, we observe three clear phases:

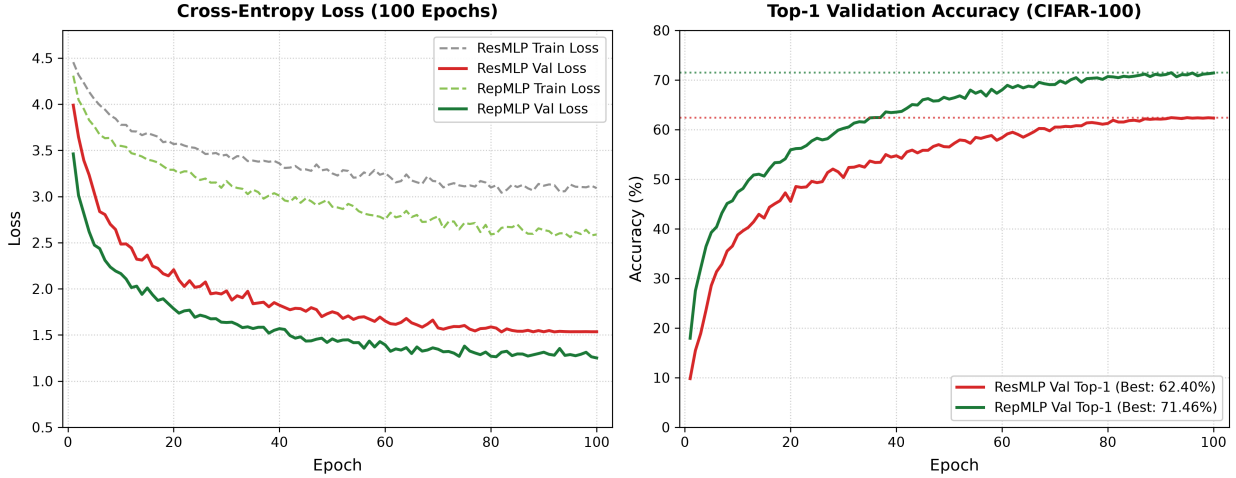


Figure 1: Training curves across 100 epochs on CIFAR-100: (Left) Cross-entropy loss on train and validation sets; (Right) Top-1 validation accuracy showing warmup, mid-phase training, and final convergence.

3.2.1. Phase 1: Early training (epochs 1–20)

During the first 20 epochs, the two models show very different representation learning speeds on both training and validation splits:

- **RepMLP warms up faster:** At epoch 1, RepMLP starts with train accuracy 7.47% (loss 4.3087) and validation accuracy 17.94% (loss 3.4612). By epoch 20, it reaches train accuracy 37.38% (loss 3.2891) and validation accuracy 55.92% (loss 1.7842). Its auxiliary 3×3 and 1×1 depthwise conv branches help the model quickly extract low-level spatial features like edges and textures.
- **ResMLP learns spatial relations from scratch:** ResMLP starts slower, with train accuracy 4.53% (loss 4.4553) and validation accuracy 9.80% (loss 3.9892) at epoch 1. By epoch 20, it reaches train accuracy 28.47% (loss 3.5670) and validation accuracy 45.52% (loss 2.2067), trailing RepMLP by -8.91% in training accuracy and -10.40% in validation accuracy. Because its linear token mixer $\mathbf{W}_{\text{token}} \in \mathbb{R}^{64 \times 64}$ has no built-in notion of 2D locality, it must discover spatial adjacency purely from gradient updates.

3.2.2. Phase 2: Mid-phase training (epochs 20–60)

Between epochs 20 and 60, both models train under heavy data augmentations (Mixup $\alpha = 0.8$, CutMix $\alpha = 1.0$, and RandAugment):

- **Impact of data augmentation:** Because Mixup and CutMix blend pairs of images and labels, the training targets are soft and mixed, making training accuracy lower than validation accuracy (evaluated on clean, unmixed images).
- **Loss and accuracy progression:** Both models lower their losses steadily. By epoch 50, RepMLP reaches train accuracy 51.48% (loss 2.8888) and validation accuracy 66.14% (loss 1.4571). In comparison, ResMLP reaches train accuracy 39.29% (loss 3.2501) and validation

accuracy 56.50% (loss 1.7507). RepMLP maintains a consistent advantage of $\approx 12.19\%$ on train accuracy and $\approx 9.64\%$ on validation accuracy.

- **Training stability:** Gradient clipping at maximum norm 1.0 effectively prevents gradient explosions, ensuring smooth optimization throughout these augmented epochs.

3.2.3. Phase 3: Final convergence (epochs 60–100)

In the last 40 epochs, the Cosine Annealing schedule smoothly decays the learning rate towards 10^{-6} :

- **RepMLP peak performance:** At epoch 92, RepMLP reaches its peak validation accuracy of **71.46%** (val loss 1.2804, train acc 62.83%), with **90.96%** Top-5 accuracy. By epoch 100, it finishes with train accuracy 63.43% (train loss 2.5888) and validation accuracy 71.40% (val loss 1.2506).
- **ResMLP peak performance:** ResMLP peaks at **62.40%** validation accuracy at epochs 92 and 95 (val loss 1.5331, train acc 43.96%), with **87.52%** Top-5 accuracy at epoch 99. By epoch 100, it finishes with train accuracy 44.72% (train loss 3.0903) and validation accuracy 62.32% (val loss 1.5347).
- **Comparison of fitting capability:** The final gaps of **+18.71%** in train accuracy (63.43% vs. 44.72%) and **+9.06%** in validation accuracy (71.46% vs. 62.40%) show that RepMLP is significantly better at fitting complex augmented distributions. This confirms that adding local convolutional priors helps vision models learn much more effectively under data constraints.

3.3. Computational efficiency and training time

Besides accuracy, our logs record the training runtime for both architectures:

- **ResMLP training speed:** ResMLP completed the full 100 epochs in **54.65 minutes** (≈ 32.8 seconds per epoch) on 2 GPUs. Its columnar structure is simple and fast to run on hardware without tensor reshaping overhead.
- **RepMLP training overhead:** RepMLP required **107.13 minutes** (≈ 64.3 seconds per epoch)—almost double the time of ResMLP. This overhead comes from running multiple parallel conv branches (1×1 and 3×3 with separate BatchNorm layers) and reshaping partitions at every stage.

This comparison highlights a clear trade-off: while RepMLP achieves higher accuracy thanks to local convolutions, its partitioning and multi-branch structure make training significantly slower. This observation motivates our Week 03 plan: designing an isotropic architecture (**Rep-ResMixer**) that adds re-parameterized convolutions directly into ResMLP’s linear token-mixer, aiming to gain the accuracy of convolutions while keeping the high speed of pure MLPs.

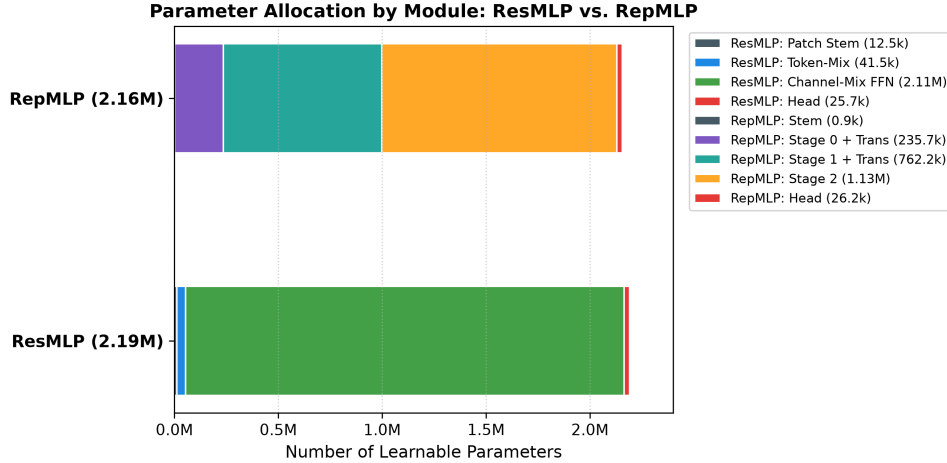


Figure 2: Parameter breakdown: ResMLP concentrates 96.36% of its parameters in channel FFNs, while RepMLP spreads capacity across hierarchical stages and multi-scale units.

4. Architectural Comparison

4.1. Isotropic columnar vs. hierarchical multi-stage architectures

Comparing ResMLP and RepMLP highlights two different design choices in modern Vision MLPs:

Dimension	ResMLP [1]	RepMLP [2]
Macro topology	Isotropic columnar (uniform $D = 256$, $N = 64$ throughout)	Hierarchical multi-stage ($16 * 16 \rightarrow 8 * 8 \rightarrow 4 * 4$)
Downsampling	Single patch embedding ($P = 4$ at stem)	Multi-step downsampling ($P = 2$ plus transitions)
Spatial mixing	Global linear matrix $\mathbf{W}_{\text{token}} \in \mathbb{R}^{N \times N}$	Partition-based FC3 with local sharesets S
Locality injection	None (pure linear token projection)	Multi-branch depthwise convolutions ($3 * 3, 1 * 1$)
Channel mixing	Affine scaling + 2-layer FFN (GELU)	Global perceptron (SE attention) + 2-layer FFN
Inference graph	Identical to training graph	Merged via structural re-parameterization

Table 4: Structural and operational comparison between ResMLP and RepMLP baselines.

4.2. How spatial tokens interact

The main difference between the two models is how they handle spatial relationships between image patches:

- **ResMLP global linear mapping:** For patch sequence $\mathbf{Z} \in \mathbb{R}^{B \times N \times D}$, the token-mixing

sublayer computes:

$$\mathbf{Y}_{\text{spatial}} = \text{Affine}_2 \left(\left(\text{Affine}_1(\mathbf{Z})^\top \mathbf{W}_{\text{token}} + \mathbf{b}_{\text{token}} \right)^\top \right) + \mathbf{Z}. \quad (14)$$

Because $\mathbf{W}_{\text{token}}$ connects every patch to every other patch, it covers the whole image in one step. However, it does not know which patches are nearby: neighboring patches are treated just like patches on opposite corners of the image.

- **RepMLP partitioned communication:** RepMLP addresses this by dividing the feature map into small local patches ($h * w$) and training them alongside depthwise conv branches. While this brings strong locality, it requires reshaping and slicing tensors back and forth, which makes the model more complex than a standard MLP.

4.3. Linear additivity and re-parameterization

From replicating both models, we arrive at an important design insight:

$$\text{Linear additivity principle: } \mathcal{T}_{\text{fused}}(\mathbf{x}) = \mathbf{W}_{\text{linear}}\mathbf{x} + \text{Conv}(\mathbf{x}) = (\mathbf{W}_{\text{linear}} + \mathbf{W}_{\text{conv}})\mathbf{x} + \mathbf{b}_{\text{conv}}. \quad (15)$$

In models like MLP-Mixer, token mixing uses two linear layers with an activation in between: $\mathbf{Z}_{\text{out}} = \sigma(\mathbf{Z}\mathbf{W}_1)\mathbf{W}_2$. Because of the non-linear activation σ , we cannot add a parallel convolution branch and fold it into the weights at test time.

In ResMLP, however, Touvron et al. [1] showed that token mixing can work well using a **purely linear layer** $\mathbf{W}_{\text{token}} \in \mathbb{R}^{N \times N}$, with non-linearities placed only in the channel-mixing FFN.

This makes ResMLP an ideal fit for structural re-parameterization: because token mixing is purely linear, we can add parallel depthwise conv branches during training, and fold them into the single linear matrix at test time without any extra runtime cost.

5. Technical Challenges, Week 03 Roadmap & TA Consultation

5.1. Technical challenges resolved in Week 02

During Week 02, we ran into and fixed three main engineering issues:

1. **DataLoader memory stability:** When training for 100 epochs with heavy augmentations (RandAugment, Mixup, CutMix), PyTorch worker processes gradually caused memory fragmentation. We resolved this by setting `persistent_workers=True`, allocating fixed shared memory, and caching dataset items, which kept RAM usage stable throughout long runs.
2. **Balancing parameter counts:** To compare an isotropic model (ResMLP) with a multi-stage model (RepMLP) fairly, we adjusted layer depths and channel widths so that both models have almost the same size ($\approx 2.16\text{M} - 2.19\text{M}$ parameters, a 1.6% difference).

3. **Mixed precision training stability:** Training deep Vision MLPs from scratch in FP16 can occasionally lead to underflow during early epochs. We added dynamic loss scaling via PyTorch AMP and set gradient clipping at norm 1.0, which prevented any numerical underflow or overflow.

5.2. Week 03 roadmap

With our baseline models replicated, our plan for **Week 03** focuses on four key tasks:

- **Train the proposed hybrid model:** Train and evaluate our proposed model, which adds structural re-parameterization directly into ResMLP’s linear token-mixing layer. We will measure its accuracy gain on CIFAR-100 compared to the baselines.
- **Verify weight fusion mathematically:** Implement automated tests to confirm that folding the convolutional branches into the linear token matrix produces the exact same output:

$$\max \|\mathbf{Y}_{\text{train_eval}} - \mathbf{Y}_{\text{deployed}}\|_{\infty} < 10^{-4}. \quad (16)$$

- **Profile inference speed and latency:** Measure latency (ms), throughput (images/sec), and GPU memory usage to verify that the deployed model runs at the same speed as vanilla ResMLP without extra overhead.
- **Scale to Tiny-ImageNet:** Run experiments on Tiny-ImageNet (64 * 64 resolution, 200 classes) to see how the models perform on slightly larger images.

5.3. Questions for Teaching Assistant (TA) consultation

In MetaFormer [3], Yu et al. showed that the general architecture (separating token mixing and channel mixing) is a key factor for vision models, demonstrating that even an extremely simple token mixer like average pooling (PoolFormer) can achieve strong performance.

However, comparing the ImageNet benchmark curves of PoolFormer and ResMLP highlights a major difference in parameter efficiency:

- **PoolFormer parameter efficiency:** PoolFormer reaches 81.4% Top-1 accuracy with only $\approx 31\text{M}$ parameters and 5.0G MACs (and reaches 82.5% with $\approx 73\text{M}$ parameters and 11.6G MACs).
- **ResMLP parameter bloat:** In contrast, ResMLP requires a massive budget of $\approx 116\text{M}$ parameters (ResMLP-36) and 23.0G MACs just to reach $\approx 81.0\%$ Top-1 accuracy.

Team Hypothesis: Our team hypothesizes that this significant gap in parameter efficiency is driven by their macro-architectural design:

- **Hierarchical pyramid with conv embeddings:** PoolFormer uses a 4-stage pyramid layout. Before each stage, a convolutional embedding layer downsamples the spatial grid while expanding channel capacity, which gradually shrinks image resolution and continuously injects local inductive bias.

- **Single-scale isotropic columnar design:** ResMLP uses a single-scale columnar structure. It embeds the image only once at the patch stem and maintains a constant token count N and channel width D across all layers. Without multi-scale spatial reduction and intermediate conv embeddings, ResMLP must scale up depth and width massively to learn rich visual features, resulting in excessive parameters.

Based on this observation, we would like to consult our Teaching Assistant on the following questions:

Q1: Validity of our architectural hypothesis: From the TA's perspective, is our hypothesis sound that ResMLP's lower parameter efficiency stems primarily from its isotropic single-scale layout and the lack of multi-stage convolutional downsampling embeddings?

Q2: Direction for Week 03 modeling: To address this limitation in our research project:

- *Option A (Pyramid ResMLP):* Should we extend ResMLP into a hierarchical multi-stage architecture with convolutional downsampling layers before each stage (similar to PoolFormer or RepMLP)?
- *Option B (Isotropic Re-parameterization):* Or should we keep the isotropic columnar structure (preserving ResMLP's simple uniform design) and focus strictly on improving data efficiency by adding structural re-parameterization directly into the linear token mixer?

Which direction would the TA recommend as more scientifically impactful and feasible within the AIO Topic Team scope?

6. References

- [1] H. Touvron et al., “ResMLP: Feedforward networks for image classification with data-efficient training”, *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 45, no. 4, pp. 5314–5321, 2023.
- [2] X. Ding et al., “RepMLPNet: Hierarchical vision MLP with re-parameterized locality”, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 578–587.
- [3] W. Yu et al., “MetaFormer is actually what you need for vision”, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 819–10 829.